

DHT11 Temperature & Humidity Sensor

Raspberry Pi 4 → ThingSpeak Cloud Monitoring

Comprehensive Lab Experiment Guide

Experiment Objective

In this experiment, you will connect a DHT11 temperature and humidity sensor to a Raspberry Pi 4, read real-time environmental data using Python, and transmit that data to the ThingSpeak IoT cloud platform where it is visualised as live graphs. By the end of this guide you will have a fully functional cloud-connected IoT monitoring system.

Source Code: You can find the complete working code for this experiment on GitHub — github.com/Himalay-NRHS/IOT-rspi-4-DHT11

Components Required

#	Component	Quantity
1	Raspberry Pi 4 Model B	1
2	DHT11 Temperature/Humidity Sensor	1
3	Jumper Wires (Female-to-Female)	3
4	USB Keyboard & Mouse	1 set
5	HDMI Monitor / Display	1
6	Internet Connection (Ethernet / Wi-Fi / Hotspot)	1

Section 1 — Raspberry Pi Setup

About the Raspberry Pi: The Raspberry Pi is essentially a fully functional, credit-card-sized computer. It runs a lightweight Linux-based operating system (Raspberry Pi OS) and has a low-power CPU, USB ports, HDMI output, and GPIO (General Purpose Input/Output) pins for connecting electronic components. To operate it you will need a USB keyboard and mouse — use the ones provided in the lab. You will also need a stable internet connection to download Python libraries; connect the Pi to the lab Ethernet, Wi-Fi network, or your personal mobile hotspot before proceeding.

Step 1

Update the System

Open the Terminal on the Raspberry Pi. Run the following three commands one by one. The first two update the package list and upgrade all installed packages to their latest versions. The third installs **pip** (Python package manager) and **venv** (virtual environment support), both of which are required for the next steps.

```
sudo apt update
sudo apt upgrade -y
sudo apt install python3-pip python3-venv -y
```

Step 2

Create a Project Folder & Virtual Environment

We will create a dedicated folder called **dht11** to keep our project files organised. Inside this folder we activate a **Python virtual environment**. A virtual environment is an isolated Python workspace — any libraries installed inside it do not interfere with the rest of the system, and the correct versions are always available when you return to work on this project.

```
mkdir dht11
cd dht11
python3 -m venv myenv
source myenv/bin/activate
```

After running **source myenv/bin/activate**, your terminal prompt will change to show **(myenv)** at the beginning — this confirms the virtual environment is active. All subsequent commands in this guide must be run with this environment active.

Step 3

Install Required Python Libraries

With the virtual environment active, install the three required libraries:

```
pip install adafruit-circuitpython-dht
pip install RPi.GPIO
pip install requests
```

Library	Purpose
adafruit-circuitpython-dht	Provides the driver to read temperature and humidity data from the DHT11 sensor

RPi.GPIO	Allows Python to control the GPIO pins on the Raspberry Pi
requests	Enables Python to make HTTP requests — used to send data to ThingSpeak

Important: If the above pip commands fail with a permissions or environment error, use the following alternative command instead: `pip3 install adafruit-circuitpython-dht --break-system-packages` This flag allows installation outside the system-managed package environment and is safe to use in a lab setting.

Section 2 — Local Sensor Test

Step 4

Write the Local Test Python Script

Before connecting to the cloud, we first verify that the DHT11 sensor is working correctly and communicating with the Raspberry Pi. Create a new file named **sensor_test.py** inside the **dht11** folder and add the following code. This script continuously reads temperature and humidity from the sensor every 3 seconds and prints the values to the terminal.

```
import time
import board
import adafruit_dht

sensor = adafruit_dht.DHT11(board.D4)

while True:
    try:
        temperature_c = sensor.temperature
        humidity = sensor.humidity
        if temperature_c is not None and humidity is not None:
            print(f"Temp={temperature_c:.1f}C  Humidity={humidity:.1f}%")
    except RuntimeError as error:
        print(error.args[0])
    except Exception as error:
        sensor.exit()
        raise error
    time.sleep(3)
```

Code Explanation

- **import board** — Provides pin definitions specific to the Raspberry Pi hardware.
- **import adafruit_dht** — Imports the DHT sensor driver installed earlier.
- **adafruit_dht.DHT11(board.D4)** — Initialises the sensor on GPIO pin 4 (physical pin 7).
- **sensor.temperature / sensor.humidity** — Reads the current temperature (°C) and relative humidity (%) from the sensor.
- **RuntimeError** — The DHT11 occasionally fails to read due to signal noise. This is caught and printed without crashing the program.
- **time.sleep(3)** — Waits 3 seconds between readings. The DHT11 requires at least 1–2 s between reads.

Step 5

Hardware Connections

Understanding the GPIO Pin Layout

The Raspberry Pi 4 has **40 GPIO (General Purpose Input/Output) pins** arranged in two columns of 20 pins each. Before making any connection, you need to know how to identify the correct pins. Follow this simple orientation guide:

- **Place the Raspberry Pi flat on the table** with the USB ports (the rectangular sockets) pointing **towards you**.

- The two rows of pins will now be at the **top-right area** of the board.
- The **top-left pin** of the left column is **Pin 1** (3.3V Power — orange in the diagram).
- The **top-right pin** of the right column is **Pin 2** (5V Power).
- Pins are numbered left-to-right across each row as you go downward: 1, 2 on the first row; 3, 4 on the second row, and so on down to 39, 40.
- Refer to the GPIO pinout diagram below to identify each pin by its number and function.



Figure — Raspberry Pi 4 GPIO Pinout Diagram. USB ports face you; Pin 1 (orange, top-left) is 3.3V.

Connections to Make

Using the pinout diagram above, locate **Pin 1**, **Pin 6**, and **Pin 7** on the Raspberry Pi, then connect the DHT11 sensor as shown in the table below. Use **female-to-female jumper wires** for all three connections.

DHT11 Pin	Raspberry Pi Pin	Function
VCC	Pin 1 (3.3V)	Power Supply
DATA	Pin 7 (GPIO 4)	Data Signal
GND	Pin 6 (Ground)	Ground

Note: DHT11 pin identification: hold the sensor with the **mesh/grid side facing you**. From left to right the pins are: **VCC** (power), **DATA** (signal), **GND** (ground). Double-check every connection carefully before powering on the Raspberry Pi.

Step 6

Run the Local Test Script

Make sure the virtual environment is active (you should see **(myenv)** in your prompt). Navigate to the project folder and run the script with:

```
(myenv) admin@raspberrypi:~/dht11 $ python sensor_test.py
```

If the sensor is connected correctly, you should see output similar to the following:

```
(myenv) admin@raspberrypi:~/dht11 $ python sensor_test.py
Temp=31.7C Humidity=46.0%
Temp=31.4C Humidity=46.0%
Temp=31.2C Humidity=46.0%
Temp=31.1C Humidity=46.0%
Temp=31.0C Humidity=46.0%
Temp=30.9C Humidity=46.0%
Temp=30.9C Humidity=46.0%
Temp=30.8C Humidity=46.0%
Temp=30.6C Humidity=47.0%
Checksum did not validate. Try again.
Temp=30.5C Humidity=45.0%
Temp=30.6C Humidity=47.0%
Checksum did not validate. Try again.
Temp=30.6C Humidity=47.0%
Temp=30.5C Humidity=47.0%
Temp=30.5C Humidity=47.0%
```

Note: "Checksum did not validate. Try again." is completely normal behaviour for the DHT11 sensor. The DHT11 uses a simple 1-wire protocol that is susceptible to occasional noise on the data line. The driver detects this automatically and skips the bad reading. As long as valid readings appear in between, the sensor is working correctly. No action is required.

Section 3 — ThingSpeak Cloud Setup

What is ThingSpeak? ThingSpeak is a free IoT analytics cloud platform developed by MathWorks. It allows devices to send sensor data over the internet using simple HTTP requests, and automatically plots the incoming data as real-time graphs. It is widely used in education, research, and industry for rapid IoT prototyping.

Why are we using it here? Our goal is to go beyond a simple local terminal readout and achieve true IoT functionality — sending sensor data from the Raspberry Pi to the cloud so it can be visualised graphically from any device, in real time. ThingSpeak makes this possible with minimal configuration and without setting up a server.

Step 7

Create a ThingSpeak Account

Open a web browser on the Raspberry Pi and navigate to:

```
https://thingspeak.mathworks.com/login
```

Create a free account using any email address. You may optionally use your institution email (@sit.ac.in) if preferred. Once registered, sign in — you will be taken to the **My Channels** page as shown below.

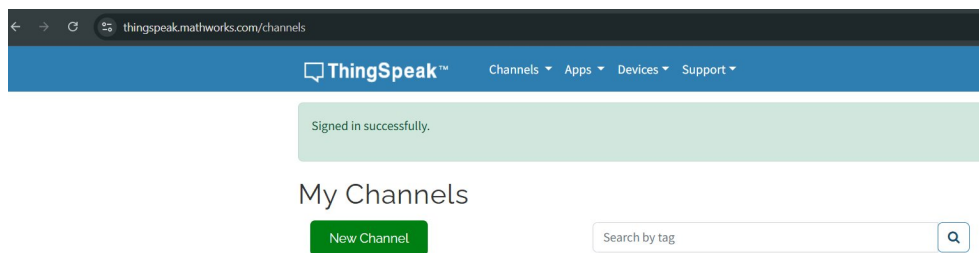


Figure 1 — ThingSpeak dashboard after successful login

Step 8

Create a New Channel

Click the green **New Channel** button. You will be taken to the channel creation form. Fill it in as follows:

- **Name:** Enter any meaningful name, e.g. *temp sensor*
- **Field 1:** Type *temperature* and tick the checkbox to enable it
- **Field 2:** Type *humidity* and tick the checkbox to enable it
- Leave all other fields blank
- Scroll down to the bottom of the page and click **Save Channel**

Figure 2 — New Channel form with Field 1 (temperature) and Field 2 (humidity) configured

Step 9

Inspect the Channel Dashboard

After saving, you will be redirected to your channel's **Private View**. You will see two empty chart widgets — one for temperature (Field 1) and one for humidity (Field 2). These will populate with live data once the Raspberry Pi starts sending readings. Now, navigate to the **API Keys** tab (indicated by the arrow below) to retrieve your write key.

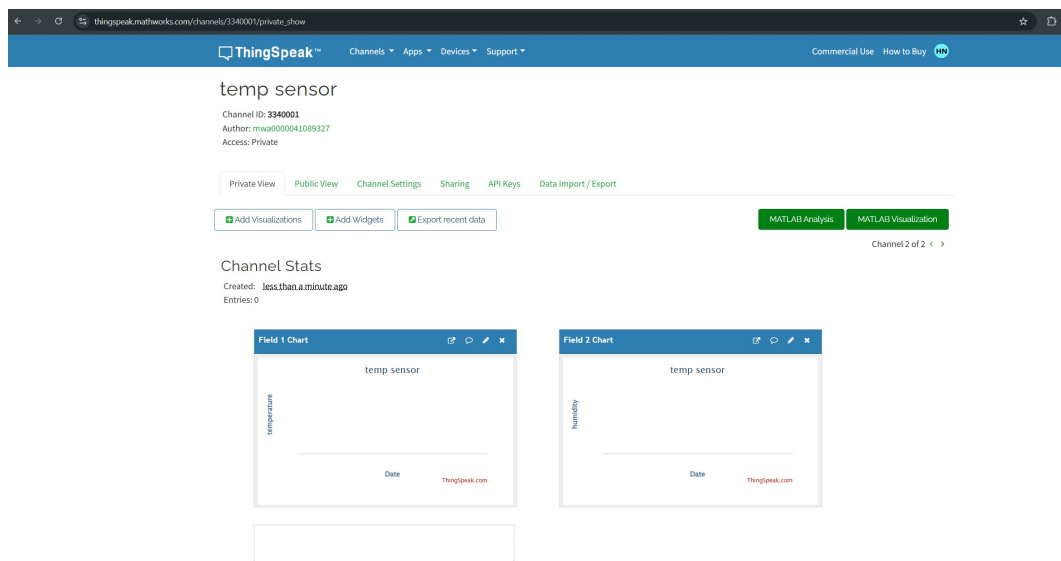


Figure 3 — Channel Private View showing empty charts; click 'API Keys' tab (arrow)

Step 10

Copy Your Write API Key

Click on the **API Keys** tab. You will see two keys — a **Write API Key** and a Read API Key. Copy the **Write API Key**. We use the Write key because our Raspberry Pi will be *writing* (sending / uploading) data to the cloud channel. Keep this key safe and do not share it publicly — it grants write access to your channel.

The screenshot shows the ThingSpeak web interface for a channel named "temp sensor" (Channel ID: 3340001). The "API Keys" tab is selected. The "Write API Key" section displays a key "OR21440K8RAWPDUIJ" with a "Generate New Write API Key" button. The "Read API Keys" section displays a key "FA5PKZ8LA52LC8SQ" with a "Note" field, "Save Note", "Delete API Key", and "Add New Read API Key" buttons. The right sidebar contains "Help" text, "API Keys Settings" instructions, and "API Requests" examples for writing, reading, and status updates.

Figure 4 — API Keys tab showing Write API Key and Read API Key (above key is a dummy sample — use your own key)

Note: The Write API Key is unique to your channel. Anyone who has this key can write data to your channel. Do not commit it to public code repositories. In this experiment, keep it in your Python script locally on the Raspberry Pi.

Section 4 — Sending Data to ThingSpeak

Step 11

Modify the Python Script to Upload Data

Create a new file named **cloud_upload.py** inside the **dht11** folder. This script extends the local test script by adding an HTTP POST request that sends each sensor reading to ThingSpeak every 15 seconds. Replace **YOUR_API_KEY** with the Write API Key you copied in the previous step.

```
import time
import board
import adafruit_dht
import requests

sensor = adafruit_dht.DHT11(board.D4)

API_KEY = "YOUR_API_KEY" # <-- Replace with your Write API Key

while True:
    try:
        temperature_c = sensor.temperature
        humidity = sensor.humidity
        if temperature_c is not None and humidity is not None:
            url = "https://api.thingspeak.com/update"
            data = {
                "api_key": API_KEY,
                "field1": temperature_c,
                "field2": humidity
            }
            requests.post(url, data=data)
            print(f"Uploaded -> Temp={temperature_c:.1f}C Humidity={humidity:.1f}%")
        except RuntimeError as error:
            print(error.args[0])
        except Exception as error:
            sensor.exit()
            raise error
        time.sleep(15)
```

Key Changes from the Local Test Script

- **import requests** — Imports the HTTP library to communicate with ThingSpeak.
- **API_KEY** — Stores your unique Write API Key as a variable.
- **url** — The ThingSpeak endpoint that accepts incoming sensor data.
- **data dict** — Contains the API key, temperature (field1), and humidity (field2) to be uploaded.
- **requests.post(..)** — Sends an HTTP POST request with the sensor data to ThingSpeak.
- **time.sleep(15)** — ThingSpeak's free tier enforces a minimum of 15 seconds between updates. Sending faster will result in requests being silently ignored.

Step 12

Run the Cloud Upload Script

It is essential to run this script from within the virtual environment. Ensure **(myenv)** is shown in your terminal prompt. If not, navigate to the project folder and activate it first:

```
cd ~/dht11
source myenv/bin/activate
(myenv) admin@raspberrypi:~/dht11 $ python cloud_upload.py
```

The terminal will print each uploaded reading. You should see output like:

```
Uploaded -> Temp=31.2C Humidity=46.0%
Uploaded -> Temp=31.0C Humidity=46.0%
Uploaded -> Temp=30.8C Humidity=47.0%
```

Step 13

View Live Graphs on ThingSpeak

While the script is running, switch back to the browser and open your ThingSpeak channel's **Private View**. Within 15–30 seconds of the first upload, the charts will begin populating with real-time data points. Field 1 will display temperature and Field 2 will display humidity, both plotted against time.

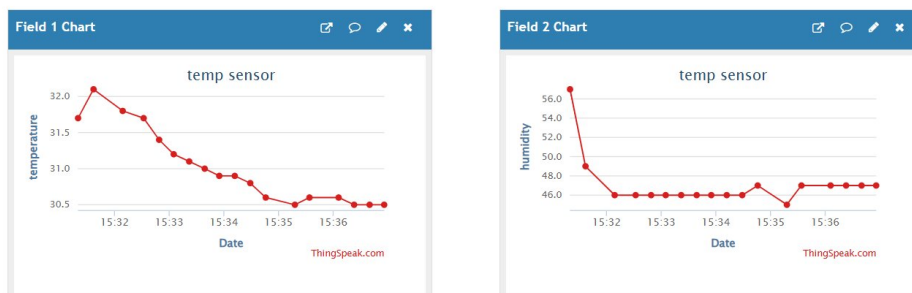


Figure 5 — ThingSpeak live graphs showing temperature (Field 1) and humidity (Field 2) data from the DHT11 sensor

Note: The graphs update automatically every 15 seconds as the Raspberry Pi sends new data. You can leave the script running for an extended period to observe trends such as temperature changes due to environmental conditions.

Conclusion

You have successfully built a cloud-connected IoT environmental monitoring system. The Raspberry Pi reads real-time temperature and humidity data from the DHT11 sensor via its GPIO pins, processes the data using Python, and transmits it to the ThingSpeak cloud platform over the internet. The data is then visualised as live time-series graphs accessible from any device with a browser.

Step	Action	Tool / Platform
1–3	System update & library installation	Raspberry Pi Terminal
4	Write local sensor test script	Python / sensor_test.py
5	Wire DHT11 to GPIO pins	Hardware
6	Verify local sensor output	Terminal
7–10	Create ThingSpeak channel & get API key	ThingSpeak Web
11	Modify script for cloud upload	Python / cloud_upload.py
12–13	Run script & observe live graphs	ThingSpeak Dashboard